

OCTOPUS Omni Cockpit — Project Instructions

Project Overview

- Next.js SaaS for Wildverse LLC / OctopusSkills (CEO: Rafael).
- Primary language: Spanish (with English i18n).
- Deployed at: `octopuskills.com` (primary) and `octopus-omni-cockpit-n8hd61.abacusai.app`.
- Both domains are **untagged** — deploy with `project_path` only (no `hostname` / `deployment_tag`).

Visual Style (Brand)

- Primary: `#C4622D` (OCTOPUS orange)
- Accent green: `#2D4A3E`
- Dark bg: `#0F1419`
- Card bg: `#1A2332`
- Border accent: `[#2D4A3E]/30`
- Standard card: `p-6 bg-[#1A2332] border-[#2D4A3E]/30`

Core Architecture

Image Models Catalog (`lib/image-models.ts`)

Shared catalog of 10 image models used by Ad Factory and UGC Factory:

- `default` (RouteLLM free)
- `openai/gpt-5.4-image-2` (NEW, max tier, 🎯, chat-image) ← `DEFAULT_IMAGE_MODEL_ID`
- `google/gemini-3-pro-image-preview` (👉 pro, chat-image)
- `google/gemini-3.1-flash-image-preview` (👉 fast, chat-image)
- `google/gemini-2.5-flash-image` (👉 std, chat-image)
- `sourceful/riverflow-v2-pro` (NEW, 🌊 pro, image-only)
- `sourceful/riverflow-v2-fast` (NEW, 🌊 fast, image-only)
- `black-forest-labs/flux.2-max` (NEW, 🔥 max, image-only)
- `black-forest-labs/flux.2-klein-4b` (NEW, 🔥 fast, image-only)
- `bytedance-seed/seedream-4.5` (NEW, 🌱 std, image-only)

Exports: `IMAGE_MODELS`, `DEFAULT_IMAGE_MODEL_ID`, `getImageModel(id)`, `groupImageModels()`, `CATEGORY_ORDER`, `CATEGORY_LABELS`, `TIER_LABELS`, types `ImageModel`, `ImageModelKind`.

Model `kind` controls backend request shape:

- `chat-image` → `modalities: ['image', 'text']` (Gemini, GPT-5.4)
- `image-only` → `modalities: ['image']` (FLUX, Riverflow, Seedream)
- `routellm` → RouteLLM fallback

Aspect Ratios

- **Ad Factory:** `1:1` (square ads)
- **UGC Factory:** `9:16` (vertical TikTok/Reels)

- Only sent when `modelMeta.supportsAspectRatio` or `supportsImageSize` is true.

OpenRouter Config Endpoint (reusable)

- GET `/api/ad-factory/config` → returns `{ hasOpenRouterKey: boolean, keyName?: string }`
- UGC Factory reuses this endpoint.

Key Pages/Features

Ad Factory (`app/(dashboard)/dashboard/ad-factory/`)

- Grouped multi-provider model selector (cards by category with NEW badges, tier badges)
- Focus Area feature (emphasizes a specific product aspect in the prompt)
- Reference image upload (up to 20 images)
- Badge showing actual model used on generated image
- `onError` handler for broken images

UGC Factory (`app/(dashboard)/dashboard/ugc-factory/`)

- Same multi-provider model selector pattern as Ad Factory
- Uses `9:16` aspect ratio instead of `1:1`
- Session storage persists state including `generatedModelEngine` and `generatedModelId`
- Does NOT have Focus Area (`frameDescription` + `backgroundPrompt` cover this need)

UGC Factory — Seedance Video Generation (3 models)

- Image-to-Video generation via `fal.ai` Queue API.
- Three models offered with different capabilities:
- `seedance_1_5_pro` : 4-12s duration, stable, MORE PERMISSIVE on real-people images. Endpoint: `fal-ai/bytedance/seedance/v1.5/pro/image-to-video`
- `seedance_2_0` : 4-15s duration, multi-shot, physics. Endpoint: `bytedance/seedance-2.0/image-to-video` ⚠ NO `fal-ai/` prefix
- `seedance_2_0_fast` : 4-15s duration, faster/cheaper. Endpoint: `bytedance/seedance-2.0/fast/image-to-video` ⚠ NO `fal-ai/` prefix
- **CRITICAL `fal.ai` endpoint pattern:** Seedance v1.5 uses legacy `fal-ai/` prefix; Seedance v2.0 (Pro and Fast) uses NEW namespace WITHOUT `fal-ai/` prefix. This is a `fal.ai` naming convention quirk — using wrong prefix causes 200 OK on POST but 404 on status polling because the `response_url` is malformed.
- For v2.0 status polling: `fal.ai` routes status URLs through PARENT namespace `bytedance/seedance-2.0/requests/<id>` for BOTH Pro and Fast. Always trust the URLs returned in the POST response.
- **Content policy gotcha (Seedance 2.0):** v2.0 has STRICTER content policies than v1.5 — rejects images containing real-people likenesses with `content_policy_violation`. `fal.ai` returns HTTP 200 + `status: COMPLETED` in this case (NOT a typical error code) but with `result.detail = [{type: 'content_policy_violation', ...}]` instead of `result.video.url`. The status route MUST detect this case and return `status: 'failed'` to the frontend with a Spanish error message recommending Seedance 1.5 Pro instead.
- Files: `app/api/ugc-factory/seedance/route.ts` (POST submit) + `app/api/ugc-factory/seedance-status/route.ts` (poll, includes `content_policy_violation` detection). Frontend at `app/(dashboard)/dashboard/ugc-factory/page.tsx` polls every 8s.
- Auth: User's `fal.ai` API key stored in `apiKey` table with `serviceType: 'ugc_fal'`.

UGC Factory — Style Mode Selector (v1, 2026-04-25)

- **Problem solved:** Seedance 2.0 / Fast (15s) blocks photorealistic person images via `content_policy_violation`. To unlock 15s video, the user MUST opt into stylized/cartoon avatars BEFORE generating the model image.
- **UI:** Step 2 of UGC Factory now starts with a **Style Mode** selector (2 cards: 🎬 Realistic / 🤖 Stylized) ABOVE the avatar selector. Includes a warning banner explaining fal.ai's policy ("Esta limitación viene de fal.ai, no de Octopus").
- **State:** `styleMode: 'realistic' | 'stylized'` in `app/(dashboard)/dashboard/ugc-factory/page.tsx`. Persisted in `sessionStorage` `SS_KEY` `ugc_factory_state_v2`. Default = `realistic`.
- **Backend:** `app/api/ugc-factory/generate-model/route.ts` accepts `styleMode` from body. When `'stylized'`, replaces photorealism rules (3, 9, 10) with explicit "STYLIZED 3D ILLUSTRATION (Pixar/Spider-Verse aesthetic), clearly NON-PHOTOREALISTIC, illustrated/animated" wording + adds anti-real-likeness rules. Saved in `creativeAsset.metadata.styleMode`. Returns `{styleMode}` in response.
- **Step 3 hint:** Shows a small color-coded reminder banner above the Seedance model selector — orange `#C4622D` if realistic (max 12s, only 1.5 Pro), emerald if stylized (up to 15s, all models work). Does NOT auto-switch user's seedanceModel selection — user picks freely.
- **i18n keys:** `ugc.style_mode_label`, `ugc.style_realistic_title/badge/desc`, `ugc.style_stylized_title/badge/desc`, `ugc.style_warning_title/body`. Both ES and EN.
- **Stylized mode prompt rules** (replace photorealism rules 3/9/10): "Pixar/Arcane/Spider-Verse aesthetic, clearly NOT photorealistic, illustrated/animated quality, lively for lip-sync, AVOID photographic realism".

OCTOPUS Chat (`app/(dashboard)/dashboard/jarvis/`) — Multi-model Image (v1, 2026-04-22)

- **Endpoint:** `/api/creative/generate` (NOT `/api/jarvis/generate-image`). Called from `jarvis/page.tsx` lines ~2941 and ~3026 via `create_media` / `generate_image` action types.
- **Backend:** `generateImage()` in `app/api/creative/generate/route.ts` accepts `imageModel` + `orientation`, uses OpenRouter when model is non-default AND user has `turbo_*` key; silent fallback to RouteLLM otherwise.
- **Frontend state:** `imageModel` in `localStorage` `octopus_chat_image_model`, default `'auto'`.
- **Three-layer model resolution** (priority descending):
 1. LLM hint (`media.imageModel` or `media.modelId` in `jarvis-action`)
 2. Auto-detect from user message via `lib/model-detector.ts` regex (nano banana, gpt-5.4, flux, riverflow, seedream, gemini)
 3. User's dropdown state (if not 'auto')
- **UI:** 🤖 button next to Turbo toggle opens modal with grouped categories (identical pattern to Ad Factory). Auto option plus all 10 models; disabled if model needs key and no OpenRouter key.
- **OCTOPUS personality** (`lib/octopus-personality.ts` `MODULE_CONTEXTS.creative`): Informed about multi-model selection, can include `imageModel` hint in `create_media` `jarvis-actions`.
- **Pre-req for non-default models:** user's OpenRouter key (reuses `GET /api/ad-factory/config` for detection).

Hero Showcase (landing)

- 4 video carousel on homepage

Important Conventions

Deployment

- Always deploy with `deploy_nextjs_project(project_path="/home/ubuntu/octopus_omni_cockpit")` — no hostname/tag.
- Both domains updated simultaneously.

i18n

- Both ES (first half ~line 800-1400) and EN (second half ~line 1900-2500) blocks in `lib/i18n-context.tsx`.
- Add new keys to both blocks in parallel locations.

Prisma + Database

- Shared DB for dev + prod.
- Avoid `--accept-data-loss` without explicit user confirmation.
- Uses `apiKey` table with `serviceType` starting with `turbo_` for all user-provided API keys (OpenRouter, etc).

TypeScript

- Verify changes with `cd nextjs_space && yarn tsc --noEmit` before testing.

Elite Creative Director Layer (v1, 2026-04-22)

Shared module: `lib/elite-creative-director.ts`

- Exports `ELITE_CREATIVE_DIRECTOR_SYSTEM` (full English, for backend system msgs)
- Exports `ELITE_CREATIVE_DIRECTOR_COMPACT` (compact Spanish, for OCTOPUS chat modular context)
- **Product-agnostic:** Instructs LLM to prioritize the USER's product (from Brand DNA/reference photos) — NOT hardcoded to Wildverse.

Mandatory output structure (for video/image prompts):

`[HOOK 0-2s]` → `[CONTRAST problem→solution]` → `[CINEMATIC DIRECTION]` (camera, lighting, rhythm)

Silent Quality Auto-Check: Would this stop the scroll on TikTok? Does it look Apple/A24 or cheap startup? → Rewrite until elite.

OCTOPUS CINEMATIC v1 Storyboard (canonical 16s reference baked into Elite Director v2, 2026-04-22):

Timed beat-sheet for ALL video/motion/demo prompts. Rafael-authored via live session; generic-ized for multi-tenant:

- [0-1.5s] HOOK (close-up + scan-line + provocative overlay <7 words + digital pulse)
- [1.5-4s] CHAOS (desaturated pain state + handheld + pain overlays)
- [4-7s] TRANSFORMATION (hard flash + color returns + scan beam + bass drop + chime)
- [7-10s] FEATURE POWER (zoom to 2-3 hero features + micro-animations)
- [10-12s] EMOTION + TRUST (★★★★★ + testimonial + calm POV)
- [12-14s] IMPACT (bold promise overlay 3-5 words + glow peak)
- [14-16s] FINAL SHOT (zoom-out + logo + 3-verb tagline + bass tail)

Plus DETAILS (shallow DoF, cut rhythm, lighting, particles), SOUND DESIGN (explicit cues per beat), GOAL (5-thought sequence: "What IS that?" → "That's my problem" → "Oh. OH." → "I trust this" → "I want it NOW").

Integration points:

1. `lib/ad-factory-templates.ts` — Elite Director prepended to `MASTER_FIDELITY_SYSTEM`
2. `app/api/motion-graphics/generate/route.ts` — Replaces both `enhanceOnly` + full refinement system msgs
3. `app/api/ugc-factory/generate-script/route.ts` — Elite Director + UGC short-form copy adaptation
4. `lib/octopus-personality.ts` — `MODULE_CONTEXTS.creative` injects compact Spanish version

Octopus Agent ALFA — FASE 1: Ollama auto-detection (v1, 2026-04-26)

Goal: Bridge local detects Ollama (HTTP + filesystem) and reports models to backend; Active Arms shows “Ollama  X models” card.

Architecture:

- The same Bridge that controls Smart Home (`/api/hogar/bridge/installer`) is now extended to detect Ollama.
- Multi-strategy detection (works even when Ollama is CLOSED):
- 1. **HTTP** → `GET http://localhost:11434/api/tags`
- 2. **Filesystem** → walks `~/.ollama/models/manifests/registry.ollama.ai/<namespace>/<model>/<tag>` (the Octopus differentiator)
- 3. **Process** → `ps -A | grep ollama` (mac/linux) or `tasklist` (windows)
- Bridge POSTs every 60s to `/api/arms/ollama/status` with `{ installed, running, models, detectionMethod, os }`.
- Backend stores the snapshot in `ArmConnection` table (`armType='ollama'`, `credentials = JSON` with status).
- Frontend (`brazos/page.tsx`) polls `GET /api/arms/ollama/status` every 30s to render “Ollama  N modelos” card with:
- Status dot:  running /  installed-but-closed /  no bridge
- Refresh button + collapsible models dropdown (top 20)
- Ollama is `isAutoDetected: true` in `ARM_CONFIGS` — has no manual “Connect” button.

Endpoints:

- `POST /api/arms/ollama/status` — Bridge auth via `x-bridge-token` (reuses `hogar_bridge ApiKey`).
- `GET /api/arms/ollama/status` — Frontend auth via session; returns `bridgePresent: false` if last report > 2 min ago (stale).

Files modified for FASE 1:

- `lib/brazos-types.ts` — added `ollama` to `ArmType`, `OllamaArmStatus` / `OllamaModel` interfaces, `ARM_CONFIGS.ollama` entry.
- `app/api/arms/ollama/status/route.ts` — new POST/GET endpoint.
- `app/api/hogar/bridge/installer/route.ts` — extended `generateBridgeScript()` with Ollama detection (`detectOS`, `ollamaHomeDir`, `ollamaInstallPaths`, `isOllamaInstalled`, `isOllamaProcessRunning`, `probeOllamaHTTP`, `listModelsFromFilesystem`, `detectOllama`, `reportOllamaStatus`); main loop now starts and polls Ollama every 60s.
- `app/(dashboard)/dashboard/brazos/page.tsx` — Ollama card with status badge, model count, dropdown, refresh.
- `lib/i18n-context.tsx` — Added `ollama.*` keys (ES + EN).
- `bridge/README.md` — documentation for the standalone Bridge directory (placeholder for FASE 3 evolution).

Octopus Agent ALFA — FASE 2: Ollama Chat (v1, 2026-04-26)

Goal: User chats with local Ollama models via Bridge relay.

Architecture — Message Queue Relay:

- User → POST `/api/arms/ollama/chat` → pending assistant msg in DB
- Bridge polls PATCH `{action:'poll'}` every 2-10s → calls local Ollama → PATCH `{action:'respond'}`
- Frontend polls GET `?action=status&messageId=X` every 1.5s

Schema (additive): `OllamaChatSession` + `OllamaChatMessage`

Bridge extension: `pollOllamaChat()` + `startChatPoll()` (adaptive 2s/10s timer), 5-min timeout per request.

UI: `/dashboard/ollama-chat` — session sidebar, model selector, optimistic UI, privacy badges.

Nav: Terminal icon + “Ollama Chat” ALFA badge. Brazos card “Chatear” button.

Code Engine ALFA — Cloud LLM brain + Local Bridge hands (v2 “Invisible IDE”, 2026-04-27)

Goal: User asks for file/code operations on their local machine. Cloud LLM (OpenRouter via Turbo Mode) generates code+commands, the Bridge executes them in the user’s workspace folder.

Design Philosophy — “Peace, not complexity” (Rafael, 2026-04-27):

Octopus is NOT a programmer’s tool. It is an EXECUTIVE execution engine. The UI must transmit calm and results, never threaten with complexity. Aesthetic reference: **Codex (OpenAI) / Apple** — pure white, soft accents, capsule input, NO terminal/hacker vibes. Code is hidden by default; the chat only confirms outcomes.

Architecture — SSE streaming + Adaptive Polling:

- Frontend → POST `/api/arms/claude-code/chat` (SSE stream) → emits `session/delta/commands/done/error` events
- LLM responses include a single ````octopus-action` fenced JSON block with shape `{ "actions": [...]} (preferred)` — parser also tolerates a single-action object or a generic ````json` fence as fallback
- Each action validated server-side (path safety, normalization). Approved commands → `BridgeCommand` table (`status='approved'` for low-risk, `'pending'` for `execute_cmd / delete_file`)
- Bridge polls GET `/api/arms/claude-code/poll` (500-800ms when commands present, 3s idle) → executes → POST `/api/arms/claude-code/respond`
- User confirms `pending` commands via POST `/api/arms/claude-code/confirm` `{commandId, decision:'approve'|'reject'}`

Strict Output Contract (LLM): System prompt enforces ONE `octopus-action` block per response, with `{"actions": [...]}` envelope. Each action uses key `"type"` (`write_file / read_file / create_dir / read_dir / delete_file / execute_cmd`). Parser normalizes `type` ↔ legacy `action` and rejects non-canonical forms.

Sandbox: All Bridge ops happen inside the user’s `octopus-workspace` folder (resolved via `path.resolve` + path traversal guards: rejects `..`, absolute paths, drive letters, home-prefixed paths).

Schema (additive): `CodeSession` , `CodeMessage` , `BridgeCommand` . User relation: `CodeSession` `CodeSession[]` .

Bridge actions supported:

- `write_file` , `read_file` , `create_dir` , `read_dir` → auto-approve (sandbox enforced)
- `delete_file` , `execute_cmd` → require user confirm

UI — “Invisible IDE” (`app/(dashboard)/dashboard/claude-code/page.tsx`):

- **Layout:** 3-column inside a single white rounded-3xl card (max-w 1200px):
- Left rail (60px): icons (logo, Folder = sessions drawer toggle, Plus = new, Settings = model)
- Center: status row → big title (current question / “¿Qué vamos a construir?”) → Activity Feed → capsule input
- Right (380px): Smart Preview pane on subtle `bg-zinc-50/50`
- **Activity Feed** replaces chat bubbles. Each command rendered as a single elegant row with stage marker:
 - ✓ completed (emerald) / ● (pulsing dark dot) executing / ○ queued or pending / ✕ failed (rose) / · rejected
- Humanized labels: “Escribiendo archivo” → “Archivo creado”, “Creando carpeta” → “Carpeta creada”, etc.
- Click any row → updates Smart Preview to that artifact
- **Code is hidden by default.** Action JSON envelope and any ````` fences are stripped from displayed prose. A “Ver detalles de la conversación” toggle reveals assistant prose if user wants it.
- **Smart Preview** (right pane) auto-renders by extension/type:
 - `.html/.htm` → sandboxed iframe with `srcDoc` + “Ver código” toggle
 - `.md/.markdown` → tinyMarkdown renderer (custom, no deps) + “Ver código” toggle
 - `.json` → pretty-printed
 - `read_dir` → file list with 📁/📄
 - `execute_cmd` → stdout (zinc-50) + stderr (rose-50) blocks
 - `create_dir` / `delete_file` → confirmation card
 - Pending → Shield icon + “Esperando tu aprobación”
 - Executing → Loader spinner + “Aplicando cambios en tu PC...”
 - Failed → AlertTriangle + sanitized one-line error (no stack traces)
 - Empty → Sparkles icon + soft hint
- **Capsule input:** rounded-full `border-zinc-200` , “Ask” button is a small dark pill. Placeholder: “Pide a Octopus que cree o ejecute algo...”
- **Friendly error fallback:** failed assistant turns show a rose-50 card with “Hubo un problema... ¿Quieres reintentar?” and a Reintentar button that re-injects the last user message into the input.
- **Sandbox notice:** tiny Shield icon + “Sandbox activo: solo se modifican archivos dentro de tu workspace.” below input.

Files:

- `lib/claude-code-prompts.ts` — `CODE_ENGINE_SYSTEM_PROMPT` (strict actions[] envelope), `parseActions` (octopus-action + json fence + single-object normalization), `validatePath` , `needsConfirmation` , `stripThinkTags` , `stripActionAndCodeBlocks`
- `app/api/arms/claude-code/{chat,poll,respond,confirm}/route.ts` (unchanged from v1 except parser benefits)
- `app/(dashboard)/dashboard/claude-code/page.tsx` — fully rewritten “Invisible IDE” (white background, Activity Feed, dual pane, Smart Preview, friendly errors, retry CTA)
- `app/api/hogar/bridge/installer/route.ts` — Bridge JS with `pollClaudeCodeCommands` / `executeBridgeCommand` / `safePath` / `startCodePoll` (unchanged)
- `lib/turbo-llm.ts` — `callLLMStream` honors explicit OpenRouter model overrides (unchanged)

Display name: “Code Engine” with ALFA badge — uses Code2 icon. Sidebar nav `/dashboard/claude-code`. i18n key `nav.codeEngine`.

Default model: `anthropic/claude-sonnet-4.6` (user can switch via dropdown to GPT-5.4, Claude Opus 4.6, DeepSeek V3.2, Gemini 3.1 Pro).

CRITICAL: Rafael MUST reinstall the Bridge from `/dashboard/hogar` so the Bridge JS includes `pollClaudeCodeCommands`. The old Bridge will not pick up Code Engine commands.

Code Engine — Sprint 4: Resiliencia e Inmortalidad (2026-04-27)

- **Offline Resilience:** Bridge detects network failure, activates offline mode. `reportResult()` replaces direct fetch calls — buffers results locally when server unreachable.
- **State Checkpoints:** `.octopus-state` file in workspace tracks current phase (idle/executing/batch/offline), last completed command, batch progress. Updated after each command execution.
- **Re-sync on Reconnect:** `flushOfflineResults()` runs on: (1) successful heartbeat after offline, (2) each poll cycle if buffer non-empty. Reports all buffered results to `/api/arms/claude-code/respond`.
- **Respond endpoint hardened:** Now accepts re-sync for already-completed commands (returns `{ ok: true, resync: true }`). Also accepts commands in ‘pending’ status (for edge cases).
- **Preview Cache:** `localStorage` caches last HTML preview. Falls back to cached version when commands unavailable. Shows “Caché” badge. Individual HTML previews also cached via `WrittenPreview`.
- **Heartbeat offline detection:** `sendHeartbeat()` now detects offline state and triggers re-sync on recovery.
- **Startup recovery:** Bridge reads `.octopus-state` on boot, logs if previous offline buffer detected.
- User must reinstall Bridge from `/dashboard/hogar` for offline features.

Code Engine — Sprint 3: Creación Autónoma y Robustez (2026-04-27)

- **Atomic Batch Transactions:** Poll endpoint returns `batch` array grouping commands by `msgId`. Bridge iterates sequentially; if one fails, remaining are reported as “skipped”. `executeBridgeCommand` returns boolean success.
- **Preview HMR (Project Preview):** `ProjectPreview` component combines all completed `write_file` commands (HTML + CSS + JS) from same folder into unified `iframe`. Auto-switches when ≥ 2 files + HTML detected. Toggle button: single-file ↔ project view.
- **CDN-First Dependency Resolution:** System prompt instructs LLM to always use CDN links (unpkg, cdnjs, esm.sh). No npm/yarn for browser projects.
- **UX:** Navigation chips on ALL completed file operations (`write_file`, `read_file`, `create_dir`, `read_dir`, `open_path`).
- **i18n keys:** `project_preview`, `project_no_html`, `project_combined`, `project_view`, `single_file`, `batch_progress`, `batch_failed`, `batch_skipped`.
- User must reinstall Bridge from `/dashboard/hogar` after deploy for batch execution to work.

Code Engine — Workspace Intelligence (Sprint 2, 2026-04-27)

Goal: Octopus always knows what's in the user's workspace without needing to ask. File Watcher detects external changes; Persistent Workspace Index gives the LLM full context.

Architecture:

- **Bridge File Watcher** (`fs.watch` recursive): Monitors `octopus-workspace` in real-time. Debounces events (1.5s) and batches them to the server.
- Ignores: `node_modules`, `.git`, `.DS_Store`, `__pycache__`, `.next`, `.cache`, `.build`, `.env`, etc.
- Fallback: if `{ recursive: true }` not supported (older Node), relies on periodic rescan.
- **Full Workspace Scan:** Runs on Bridge startup + every 2 minutes. Walks up to depth 6, caps at 2000 entries.
- **Backend Storage:**
 - `WorkspaceIndex` model (userId unique, fileTree JSON, fileCount, totalSize, rootPath, lastScanAt)
 - `FileChangeEvent` model (userId, eventType, filePath, fileSize, isDir, detectedAt)
- **API:** POST/GET `/api/arms/claude-code/workspace`
 - POST `action: 'full_scan'` — Bridge reports full tree
 - POST `action: 'file_changes'` — Bridge reports delta events (also updates tree incrementally)
 - GET `?action=index` — returns full tree for frontend
 - GET `?action=changes&since=ISO` — returns recent change events
 - GET `?action=context` — returns compact text summary for LLM injection
- **LLM Context Injection:** `chat/route.ts` auto-appends workspace context (dirs, files, extensions, recent changes) to the system message. Non-critical — silently skipped if no index exists.
- **Frontend:** Right pane has two tabs — “Preview” (existing) and “Workspace” (new). Workspace tab shows:
 - ⚡ Context Active badge + file count + total size
 - Recent external changes (with emoji icons and timestamps)
 - File tree with indentation, folder/file icons, and sizes
 - Last scan timestamp
- **i18n:** `ce.ws_*` keys in both ES and EN

CRITICAL: Rafael must REINSTALL the Bridge from `/dashboard/hogar` after this deploy so the Bridge includes the File Watcher and Workspace Scanner logic.

Code Engine — Live Preview Panel (2026-04-28)

Goal: Preview panel auto-refreshes when files change, with expand/fullscreen and live reload.

Frontend enhancements (page.tsx):

1. **Auto-refresh on completion:** `previewRefreshKey` state increments when new commands complete. Used as `key` prop on iframes to force re-mount/re-render.
2. **Auto-select latest HTML:** When commands complete, auto-selects the latest completed HTML `write_file` for preview and switches to Preview tab.
3. **Preview toolbar:** Three buttons in the tab bar:
 - 🔄 Refresh (manual refresh, increments `refreshKey`)
 - ↗ Open in browser (links to `localhost:9753/<path>` — Bridge local file server)
 - 🖥 Expand/Fullscreen (toggles `previewExpanded` state → overlay modal)
4. **“Updating...” overlay:** When any command is executing/approved, a translucent overlay with spinning indicator and “Actualizando...” text covers the preview pane.
5. **Expanded preview modal:** Full `85vh` height, `max-w-5xl`, with its own toolbar (refresh, open in

browser, project/single toggle, close). Uses same `PreviewPanel` component with shared `refreshKey`.

6. **Pulsing dot on tab:** Preview tab shows amber pulsing dot when commands are executing.

Bridge local file server (installer/route.ts):

- `startPreviewServer()` : HTTP server on port 9753 serving files from `octopus-workspace`.
- Injects auto-reload WebSocket script into HTML files.
- WebSocket server on port 9754 : sends "reload" frame when file changes detected.
- `fs.watch(recursive: true)` monitors workspace for `.html/.css/.js/.json/.svg` changes, debounces 400ms.
- MIME type mapping for 20+ common extensions.
- Port fallback: if 9753 in use, tries 9755.
- Sandboxed via `safePath()` — no path traversal.

i18n keys added: `ce.preview_refresh`, `ce.preview_open_browser`, `ce.preview_expand`, `ce.preview_collapse`, `ce.preview_updating` (ES + EN).

CRITICAL: Rafael must REINSTALL the Bridge from `/dashboard/hogar` for the local file server to work. Without it, "Open in browser" won't work but the in-app preview still functions normally.

Code Engine — Sprint 8: Multi-File Atomic Transactions & Rollback (2026-04-28)

Goal: When Octopus generates multiple file operations, they execute as a single atomic unit. If any validation fails, all changes roll back — workspace is never left in a broken state.

Architecture — Transaction Manager (`lib/transaction-manager.ts`):

- `beginTransaction(stagedCommands)` → creates Transaction with unique txId, holds StagedCommands in memory
- `validateTransaction(tx, existingFilePaths, existingFileContents)` → runs Code Intelligence on entire batch
- `commitTransaction(tx, prisma, sessionId, messageId, withDbRetry)` → bulk creates BridgeCommands in DB
- `rollbackTransaction(tx, reason)` → returns error details; nothing written to DB

Flow: Stage → `transaction_started` → Validate → `transaction_validating` → (valid? commit : rollback) → done

SSE Events: `transaction_started`, `transaction_validating`, `transaction_committed`, `transaction_rolled_back`

Frontend: `txStatus` / `txDetails` state, animated banners (sky→violet→emerald or red), auto-fade 8s on commit.

i18n: `ce.tx_started`, `ce.tx_validating`, `ce.tx_committed`, `ce.tx_rolled_back`, `ce.tx_critical_errors`, `ce.tx_affected_files`

Files: `lib/transaction-manager.ts` (NEW), `chat/route.ts` (refactored), `page.tsx` (SSE + UI), `i18n-context.tsx`

Code Engine — Sprint 11: Phoenix Protocol (Snapshot/Rollback/Integrity, 2026-04-28)

- **Bridge Snapshot Engine** in `installer/route.ts` (~lines 567-768): `createSnapshot`, `rollbackFromSnapshot`, `verifyIntegrity`, `purgeOldSnapshots`, `listSnapshots`, `markCheckpoint`, `getLatestSnapshot`, `reportSnapshotEvent`
- **executeBatch()** wraps with: `createSnapshot` → `execute` → `verifyIntegrity` → `auto-rollback` if failed mid-batch
- **New Bridge commands:** `rollback`, `list_snapshots`, `save_checkpoint`
- **Server:** `/api/arms/claude-code/snapshots/route.ts` — event store + frontend action relay
- **Pipeline:** 9 stages: `llm` → `parse` → `validate` → `deps` → **snapshot** → `commit` → `bridge` → **verify** → `feedback`
- **Frontend:** Phoenix toast (🔥), SNAP/VERIFY telemetry pills, natural language rollback detection (“deshaz”, “rollback”, “undo”, “vuelve al estado anterior”)
- **i18n:** 6 Phoenix keys (`ce.phoenix_*`) in ES + EN
- **System prompt:** Phoenix Protocol section in `claude-code-prompts.ts`
- Snapshots local at `~/octopus-workspace/.octopus-snapshots/`, FIFO 10, named checkpoints immune
- Bridge must be reinstalled after deploy

Code Engine — Sprint 10: The Nervous System (Total Context Integration, 2026-04-28)

Goal: Transform passive capabilities into active, reactive context for the LLM. Octopus now “feels” its environment.

Core module: `lib/nervous-system.ts`

- `formatExecutionFeedback(results)` → formats [SYSTEM_FEEDBACK] block with command results
- `gatherExecutionResults(prisma, sessionId, messageId)` → collects completed/failed command results from DB
- `getWorkspaceDelta(prisma, userId, since)` → computes new/modified/deleted files since timestamp
- `formatWorkspaceDelta(delta)` → [WORKSPACE_DELTA] block for LLM
- `buildFileDiffs(paths, existingContents)` → creates file previews (first 5 + last 3 lines + metrics)
- `formatDiffsForLLM(diffs)` → [EXISTING_FILES] block for surgical editing
- Telemetry: `createTelemetryPipeline()` / `updateTelemetryStage()` / `formatTelemetryForSSE()` — 7-stage pipeline

4 context layers injected into system prompt (`chat/route.ts`):

1. [SYSTEM_FEEDBACK]: Previous execution results (success/fail/error/stdout)
2. [WORKSPACE_DELTA]: Files changed since last user message
3. [EXISTING_FILES]: Smart diff previews of files about to be overwritten
4. Existing: [WORKSPACE CONTEXT], [DEPENDENCY GRAPH]

Live State Injection (`respond/route.ts`): Auto-creates [SYSTEM_FEEDBACK] system message when all commands complete.

Telemetry Pipeline (7 stages): LLM → PARSE → AST → DEPS → DISK → BRIDGE → FB. SSE `telemetry` event. Frontend collapsible monitor.

i18n: `ce.feedback_injected`, `ce.nervous_system` (ES + EN)

Complete Pipeline (Sprints 1-10): LLM → parseActions → Stage → BEGIN tx → VALIDATE → SCAN deps → RESOLVE CDN → INJECT HTML → LOCK → COMMIT → Bridge → FEEDBACK → next LLM call

Code Engine — Sprint 9: Dependency Resolution Engine (FINAL SPRINT, 2026-04-28)

Goal: When Octopus generates code with external library imports, the system automatically detects, resolves via CDN, auto-injects into HTML, and generates a lock file — all before commit.

Core module: `lib/dependency-resolver.ts`

- `CDN_REGISTRY` : 40+ popular packages mapped to CDN URLs (React, Three.js, GSAP, Chart.js, Phaser, D3, Bootstrap, Tailwind, p5, etc.) organized by category (framework, 3d, animation, ui, data-viz, game, media, utility)
- `scanDependencies(stagedCommands, existingFileContents)` → scans all imports/requires/CDN refs, categorizes npm vs local vs CDN, detects browser vs Node.js project type
- `injectCDNIntoHTML(html, deps)` → auto-injects `<script>` / `<link>` CDN tags (CSS before `</head>`, JS before `</body>`), avoids duplicates
- `generateLockFile(deps)` → creates `octopus.lock` content with versions/CDN URLs grouped by category
- `applyDependencyResolution(stagedCommands, scanResult)` → updates staged HTML files + adds lock file + npm install commands for Node.js projects
- `formatScanForSSE(scan)` → formats scan result for SSE event payload
- **esm.sh fallback:** Unknown packages auto-resolved via `https://esm.sh/<package>@latest`

Integration (`chat/route.ts`): Inserted between VALIDATE and COMMIT in transaction flow:

1. `dependency_scanning` SSE → 2. `scanDependencies()` → 3. `dependency_resolved` SSE → 4. `applyDependencyResolution()` → 5. `dependency_ready` SSE → 6. COMMIT

Frontend (`page.tsx`): `depStatus` + `depDetails` state, animated banners:

- Scanning: blue spinner
- Resolved: amber, package pills with name+version, injected files list
- Ready: emerald checkmark, auto-fade 10s

System prompt (`claude-code-prompts.ts`): Updated `DEPENDENCY RESOLUTION` section to `CDN-FIRST + AUTO-RESOLVER`, informs LLM about auto-resolver capabilities.

i18n: `ce.dep_scanning`, `ce.dep_resolved`, `ce.dep_ready`, `ce.dep_injected`, `ce.dep_lock`, `ce.dep_install` (ES + EN)

Complete Code Engine Pipeline (all 9 sprints): LLM → parseActions → Stage → BEGIN tx → VALIDATE (Code Intelligence) → SCAN deps → RESOLVE CDN → INJECT HTML → LOCK file → COMMIT → Bridge executes

Code Engine — Sprint 7: Code Intelligence (AST + Dependency Graph, 2026-04-28)

- **lib/code-intelligence.ts** — Core engine: `parseFile` (regex-based for JS/TS/JSX/TSX/HTML/CSS/JSON), `buildDependencyGraph`, `validateFileImports`, `formatGraphForLLM`
- **app/api/arms/claude-code/analyze/route.ts** — POST (single file analysis), GET (full session dependency graph)

- **chat/route.ts** — Injects [DEPENDENCY GRAPH] into LLM system prompt; post-write validation with `dependency_graph_ready` + `code_warnings` SSE events
- **page.tsx** — `depGraph` + `codeWarnings` state; sky-blue graph badge, amber warning banners
- **i18n**: `ce.dep_graph_ready`, `ce.code_warnings`

Code Engine — Sprint 6: HMR Preview Engine (2026-04-28)

- **extractProjectFiles(commands)**: Deduplicates `write_file` commands (last-write-wins), includes `approved` / `executing` statuses for instant preview
- **buildProjectHtmlFromFiles(fileMap)**: Adds `data-hmr-file` attributes on `<style>` and `<script>` tags, injects HMR runtime
- **HMR Runtime**: Injected into `iframe`, listens for `postMessage` (`hmr:css` , `hmr:js`), updates DOM in-place, sends `hmr:ack` back
- **ProjectPreview rewrite**: `useRef` for `iframe/prevFiles/hmrReady`, `useState` for `liveHtml`, CSS-only → `postMessage` hot swap, JS-only → script reload, HTML/structural → full `srcDoc` rebuild
- **Sync indicator**: ⚡ Zap icon pulses green on HMR updates, “Listo”/“Ready” when idle
- **Auto-detect**: Triggers on `approved/executing` commands (not just completed)
- **i18n**: `ce.hmr_idle` (ES: “Listo”, EN: “Ready”)

Code Engine — Sprint 5: Session Memory (Persistent Cross-Session Context, 2026-04-28)

Goal: Octopus remembers previous coding sessions. When user returns, relevant context from past sessions is auto-injected into the system prompt.

Architecture — LLM-based (no pgvector):

- `pgvector` NOT available on managed DB. Instead:

1. **Summary generation**: LLM (`gpt-4.1-mini`) extracts structured summary from session conversation
2. **Relevance scoring**: Two-tier: keyword matching (fast path) → LLM scoring (ambiguous cases)
3. **Context injection**: Top 3 relevant memories injected into system prompt as `[SESSION MEMORY]` block

Schema (additive):

- `SessionMemory` model:
 - `id`, `userId`, `sessionId` (unique), `summary`, `filesCreated`, `decisions`, `stackUsed`, `userPreferences`, `tags`, `messageCount`, `commandCount`, `createdAt`
 - All array fields stored as JSON strings
 - Index: `[userId, createdAt]`
- `CodeSession.summarized` Boolean field added (default false)
- `CodeSession.sessionMemory` optional relation

Flow:

1. User starts new Code Engine session → `summarizeRecentSessions()` fires in background (non-blocking)
2. `retrieveRelevantMemories()` called with first message → returns `MemoryContext` with injected-Prompt

3. System prompt gets `[SESSION MEMORY]` block appended with summaries of relevant past sessions
4. SSE event `session_memory_loaded` emitted → frontend shows violet indicator for 6s
5. Previous sessions marked `summarized: true` after summary generated

Files:

- `lib/session-memory.ts` — Core engine: `generateSessionMemory`, `retrieveRelevantMemories`, `summarizeRecentSessions`, `getUserMemories`, `buildMemoryPrompt`
- `app/api/arms/claude-code/memory/route.ts` — GET endpoint with `?action=all|search|summarize`
- `app/api/arms/claude-code/chat/route.ts` — Modified: imports `session-memory`, auto-summarizes on new session, retrieves+injects memory, emits SSE event
- `app/(dashboard)/dashboard/claude-code/page.tsx` — `memoryLoaded` state + indicator banner + SSE handler for `session_memory_loaded`
- `lib/i18n-context.tsx` — `ce.memory_loaded` key (ES + EN)
- `prisma/schema.prisma` — `SessionMemory` model, `CodeSession.summarized` field

i18n: `ce.memory_loaded` — “Memoria de sesiones anteriores cargada — Octopus recuerda tu contexto”

Future phases (planned, NOT implemented yet):

- FASE 4: Code Workspace (editor + terminal + preview, multi-file)
- FASE 5: Final UI (arm selector, model selector across phases)

Code Engine — Action Parser Fix: Unfenced JSON (2026-04-27)

Root cause: Claude Sonnet 4.6 sometimes outputs raw JSON `{ "actions": [...] }` WITHOUT wrapping in `````octopus-action` code fences, especially for large multi-file payloads. The parser only matched fenced blocks → 0 actions extracted → Bridge had nothing to execute → user saw raw JSON in chat.

Fix: Added 3-tier fallback in `parseActions()` (`lib/claude-code-prompts.ts`):

1. `````octopus-action` fence (preferred)
2. `````json` fence (fallback)
3. **NEW:** Raw unfenced JSON with brace-counting parser (handles nested braces in `content` fields)

Also added:

- `withDbRetry()` wrapper for `BridgeCommand` creation in `chat/route.ts` (handles idle-session timeouts)
- Detailed logging: `[parseActions]` Found N actions and `[CodeEngine]` Created command: X id=Y
- `stripActionAndCodeFences()` in `page.tsx` now also strips raw unfenced JSON from the displayed prose
- Console logging for debugging action parsing failures

Code Engine — Real-time Execution Visibility (2026-04-27)

Added `ExecutionBanner` component showing:

- **Redesigned v2:** Light bg with colored left border accent (green=completed, amber=executing, blue=analyzing, rose=failed)
- “PROGRESS X/Y” label + full-width progress bar + “XX%” percentage
- Color-coded status: green for completed, amber for in-progress, blue for thinking, rose for failed
- Shimmer animation on progress bar during thinking phase

- **Always visible** when session has commands (not just during execution)
-  Completed /  Ejecutando /  Analizando /  Error status badges

ActivityRow numbered steps: Each command shows a numbered circle (w-6 h-6 rounded-full) with:

- Green bg + white checkmark for completed
- Amber pulsing dot for executing
- Zinc outline for queued
- Rose bg for failed
- Old `StageMark` (○/●/✓/×) replaced by numbered step circles

Terminal output redesigned from hacker-style (dark bg, green text) to Apple-aesthetic (light bg-zinc-50, zinc-600 text, rounded borders). Now visible for BOTH executing AND completed commands.

Assistant text responses now always visible after completion (not just when there are no commands).

Uses `stripActionAndCodeFences()` to clean the display.

i18n keys added: `ce.terminal_output`, `ce.phase_analyzing`, `ce.phase_executing`, `ce.phase_completed`, `ce.phase_failed`, `ce.phase_files`, `ce.phase_progress` (ES + EN).

Code Engine — Model Persistence Fix (2026-04-27)

Problem: User selects a model in Code Engine, but on page reload or session switch the model reverted to default.

Fix — 3-layer persistence:

1. **localStorage** (`ce_selected_model`): Written on every model change. Read on initial load as fall-back.
2. **fetchSession restoreModel param:** When loading an existing session, `fetchSession(id, true)` reads `session.model` from DB and sets dropdown. New messages use `fetchSession(id, false)` to avoid overwriting user's current selection.
3. **DB update on send:** `POST /api/arms/claude-code/chat` now updates `codeSession.model` with the selected model on every message, so the session always remembers the last-used model.

LLM model mapping fix: `anthropic/claude-sonnet-4.6` → actual OpenRouter model `anthropic/claude-sonnet-4-6` (dot vs dash). Mapping in `lib/turbo-llm.ts` `callLLMStream`.

Code Engine — Smart Navigation / Open Path (v3, 2026-04-27)

Problem solved: After Code Engine creates files in the user's workspace, the user had to manually search for them in their PC. Now Octopus offers direct CTAs to open files/folders in the native explorer.

New Bridge action: `open_path`

- Bridge installer (`/api/hogar/bridge/installer`) handles new case in `executeBridgeCommand` switch:
- Windows → `explorer.exe "<path>"` (does NOT reject on non-zero exit; explorer returns 1 on success)
- macOS → `open "<path>"`
- Linux → `xdg-open "<path>"`
- Sandboxed via `safePath()`. Empty path resolves to workspace root.
- Validates path exists with `fs.statSync` before launching.
- LLM is allowed to emit `open_path` actions in the strict octopus-action contract.
- Backend validation in `chat/route.ts` allows empty path for `open_path` (= workspace root) and `execute_cmd`.

New endpoint for direct UI button clicks (no LLM): `POST /api/arms/claude-code/open`

- Body: `{ sessionId, path }`. Auth via session.
- Verifies session ownership + Bridge online status.
- Creates `BridgeCommand` with `type='open_path', status='approved'` (auto-approved, low-risk action).
- Bridge picks it up on next poll.

UI — Action chips on completed rows (`ActivityRow`):

- Shows below row when command is `completed` AND `payload.path` exists for `write_file / create_dir / read_dir`:
- “↗ Abrir archivo” (only for `write_file`) → opens the file
- “📁 Abrir carpeta” → opens parent folder (`write_file`) or self (`create_dir/read_dir`)
- “👁 Ver preview” (only for HTML `write_file`) → selects command for right-pane preview

UI — Project Summary card (renders below activity feed):

- Heuristic: detects 2+ `completed write_file` commands sharing the same root folder.
- Shows file tree visualization in mono font (📁 root/ ├── file1 └── file2).
- CTAs: “📁 Abrir carpeta” (always), and if HTML detected: “👁 Ver preview” + “↗ Abrir en navegador”.
- Helper: `parentDir(p)` extracts parent folder; `isHtmlPath(p)` matches `.html / .htm`.

CRITICAL: Rafael must REINSTALL the Bridge from `/dashboard/hogar` after this deploy so the new `open_path` handler is included in the Bridge JS. Without reinstalling, “Abrir carpeta” buttons will appear to do nothing (the Bridge will receive the command but not know how to execute it).

Code Engine — Sprint 13: Visual Powerhouse (2026-04-29)

Goal: Every landing page generated by Code Engine looks like a \$50K agency build automatically.

4 features implemented:

Feature 1: CSS Premium Recipes (System Prompt)

- Added `PREMIUM VISUAL RECIPES` section to `CODE_ENGINE_SYSTEM_PROMPT` with mandatory recipes:
- Color & depth (dark bg, radial gradients, gradient text)
- Glassmorphism (backdrop-filter, scroll-reactive header)
- CSS Particles (15-20 floating divs, `@keyframes float`)
- Card hover effects (`translateY`, border-color transition, box-shadow)
- Button glow (gradient, hover shadow)
- Typography rules (Inter, clamp sizes, font weights)
- Responsive rules
- Hero image rules (use `generate_image`, never external URLs)

Feature 2: Auto Scroll Animations

- **Module:** `lib/visual-enhancer.ts`
- `injectScrollAnimations(html)` — adds CSS + IntersectionObserver JS to HTML files
- `applyVisualEnhancements(stagedCommands)` — processes all `write_file` commands in pipeline
- `detectBrokenImageUrls(html)` — scans for known-broken image domains (Spline CDN, placeholder services)
- **Pipeline integration** (`chat/route.ts`): Runs AFTER dependency resolution, BEFORE image generation extraction

- SSE events: `visual_enhanced` (files enhanced), `broken_images_warning` (broken URLs detected)
- **Auto-detect:** Also adds `animate-on-scroll` class to `<section>` tags + auto-discovers cards/features/FAQ items
- System prompt tells LLM to add `class="animate-on-scroll"` — combined with auto-injection = double coverage

Feature 3: generate_image for Heroes

- System prompt now has  `HERO_IMAGES` section explicitly forbidding external URLs (CORS/hotlink protection)
- Recommends CSS mesh gradients for decorative backgrounds
- Broken image URL detection warns when LLM uses known-problematic domains
- Existing Sprint 12 image generation pipeline handles the actual generation

Feature 4: Section-Based Editing

- System prompt now has `SECTION-BASED EDITING` section with rules:
- Read file first, identify target section, rewrite ONLY that section
- Section detection: HTML comments `<!--SECTION:name-->`, `<section id="...">`, semantic HTML5
- When writing NEW HTML: wrap sections with `<!--SECTION:header-->...<!--/SECTION:header-->` markers
- 6 standard markers: header, hero, features, pricing, faq, footer

Files modified:

- `lib/claude-code-prompts.ts` — Added Premium Visual Recipes + Section-Based Editing sections
- `lib/visual-enhancer.ts` — NEW module
- `app/api/arms/claude-code/chat/route.ts` — Visual enhancer + broken image detection integration
- `app/(dashboard)/dashboard/claude-code/page.tsx` — `visual_enhanced` SSE handler

Code Engine — Phase 1: In-Browser React Transpilation (2026-04-30)

New file: `lib/framework-preview.ts` (~250 lines)

- `buildFrameworkPreview(fileMap: Map<string,string>)` → self-contained HTML string or null
- Only activates for React projects (checks `deps['react']` in `package.json`)
- **Babel standalone** transpiles JSX/TS → JS in-browser via `<script type="text/babel" data-type="module">`
- **esm.sh import maps** with scoped deduplication ensure single React instance
- **Tailwind CDN** handles utility classes; `@tailwind` directives stripped from CSS
- **@fontsource** imports auto-converted to Google Fonts `<link>` tags
- Flattens all local files into one module (sorted: `utils` → `components` → `App` → `main`)
- Strips local imports (code is inlined), keeps npm imports (resolved via import map)
- Escapes `</script` → `<\script` in code to prevent premature tag closing in `srcDoc`
- Loading spinner + error boundary handler built into generated HTML

Modified: `page.tsx` — `frameworkPreviewHtml` `useMemo` calls `buildFrameworkPreview`

- Priority: framework preview iframe → smart card fallback → normal HTML preview
- Violet badge overlay “⚡ Vite + React — Live Preview”
- iframe with `sandbox="allow-scripts allow-same-origin allow-modals"`

Phase 2: Console & Error Overlay (2026-04-30):

- Iframe injects console capture: overrides `console.log/warn/error/info` → `par-`

```
ent.postMessage({type: '__oc_console', ...})
```

- Error overlay inside iframe: dark panel at bottom with × dismiss, auto-appears on runtime errors
- `window.onerror` + `unhandledrejection` both captured and forwarded
- `ConsolePanel` component in `page.tsx`: collapsible DevTools-lite below iframe
- Auto-opens on first error, auto-scrolls, color-coded levels (log/info/warn/error)
- Error count badge (red, pulsing) + warn count badge on header bar
- Clear button, max 200 entries with ring buffer
- Entries cleared automatically on new build (`frameworkPreviewHtml` change)
- `postMessage` listener on parent window, filtered by `type === '__oc_console'`

Phase 3: React Router + Multi-Page (2026-04-30):

- Auto-detects `react-router-dom` in deps or imports
- Patches `BrowserRouter` / `MemoryRouter` → `HashRouter` in transpiled code (`srcdoc` needs hash routing)
- Ensures `HashRouter` in import map with scoped React deduplication
- Route bridge: iframe reports hash changes via `parent.postMessage({type: '__oc_route', path})`, polls 10x after load
- Parent sends navigation commands via `postMessage({type: '__oc_navigate', path})`
- `MiniUrlBar` component: framework badge + back/home/refresh buttons + editable URL input
- Shows current route from iframe, click to edit, Enter to navigate, Escape to cancel
- Only appears when router is detected (`frameworkPreviewResult.hasRouter`)
- When no router: falls back to violet badge overlay (same as before)
- `buildFrameworkPreview` now returns `FrameworkPreviewResult { html, hasRouter }` instead of plain string

Phase 4: Hot Reload Incremental (HMR) (2026-04-30):

- Edit a file → only re-transpile that module, no full iframe rebuild. Preserves React state (like Vite HMR).
- `classifyFileChanges(prev, curr) → 'none' | 'css' | 'js' | 'structural'`
- CSS: `postMessage({kind: 'css', css}) → #__app_css` textContent swap. Console: [HMR] ✨ CSS updated
- JS: `postMessage({kind: 'js', code, globals}) → Babel.transform + __oc-`
- `Root.render(createElement(__ocApp))`. Console: [HMR] ⚡ Module updated – React state preserved
- Structural: `fwIframeKey` increment → full rebuild
- Module cache: `window.__ocModules` via `generateModuleRegistration()`. HMR globals via `generateHMRGlobals()`
- Root capture: `window.__ocRoot = createRoot(el)`, `window.__ocApp = RootComponent`
- `FrameworkPreviewResult.hmr: { css, code, globals, rootComponent }`
- `page.tsx`: `fwPrevFilesRef` + `fwIframeKey` (prefixed to avoid collision with ProjectPreview HMR)

Phase 5: Multi-Framework Preview (2026-04-30):

- `lib/framework-preview.ts` refactored into modular architecture (~1055 lines):
- **Shared utilities** extracted: `loadingOverlay(label)`, `CONSOLE_CAPTURE_SCRIPT`, `CSS_HMR_SCRIPT`, `REMOVE_LOADER`, `buildNpmImportMap(deps, extra?)`
- **DetectedFramework** type: `'react' | 'vue' | 'svelte' | 'vanilla'`
- `detectFramework(deps, files)`: Priority: react → vue → svelte → vanilla. Checks deps first, then file extensions (`.vue/.svelte`), falls back to vanilla for `.js/.ts`
- `buildFrameworkPreview`: Router that parses `package.json`, calls `detectFramework`, delegates to framework-specific builder
- `FrameworkPreviewResult` now includes `framework: DetectedFramework` field

- **Vue 3 Preview** (`buildVuePreview`):

- Parses `.vue` SFCs via regex (`<template>` , `<script>` , `<style>`)
- Supports `<script setup>` (wrapped in `setup()` function) and Options API
- Uses `vue.global` build from `esm.sh` (includes runtime template compiler — no `@vue/compiler-sfc` needed in browser)
- `vue-router` support with `createWebHashHistory()` (same hash-based approach as React Router)
- CSS HMR via shared `CSS_HMR_SCRIPT`

- **Svelte Preview** (`buildSveltePreview`):

- Compiles `.svelte` files in-browser via `svelte/compiler` from `esm.sh`
- Blob URL + dynamic `import()` pattern for compiled output
- Supports Svelte 4 (`new Component({target})`) and Svelte 5 (`mount(Component, {target})`)
- CSS HMR via shared `CSS_HMR_SCRIPT`
- Router: not supported yet (no standard Svelte router detected)

- **Vanilla JS Preview** (`buildVanillaPreview`):

- Flattens JS/TS modules via `processCodeFile` , merges HTML body content
- Import maps for npm deps via `esm.sh`
- CSS HMR via shared `CSS_HMR_SCRIPT`

- **page.tsx changes:**

- `frameworkPreviewResult` `useMemo` no longer gated on `frameworkInfo` — activates for any files
- Badge overlay shows framework-specific name: Vue 3 / Svelte / Vanilla JS / React
- `frameworkInfo` references use optional chaining (`?.`) since it can be null

Phase 6: Asset Pipeline (2026-05-01):

- Makes Code Engine previews pixel-perfect with all assets: images, icons, fonts, mock APIs
- **AssetContext interface:** { `images: Map<string,string>`, `mockEndpoints: Map<string,string>`, `svgDataUris: Map<string,string>` }

- **Image URL Rewriting** (two-tier):

1. `rewriteCodeImagePaths(code, assets)` — pre-processes code strings, replaces image path references in string literals with actual blob/CDN URLs BEFORE HTML injection (static, compile-time)
2. `buildAssetPipelineScript(assets)` — runtime `MutationObserver` + `DOMContentLoaded` + deferred runs. Rewrites `<img src>` , `background-image: url()` , `<source srcset>` . Maps path variants (with/without slash, filename-only)

- **Icon Library CDN Auto-Detection** (`detectIconCDNs`):

- `ICON_LIBRARY_CDNS` : 12 icon libraries (lucide, fontawesome, heroicons, mui, react-icons, bootstrap-icons, material-icons, ionicons, phosphor, tabler)
- Dual detection: (1) check deps/imports for library names, (2) scan code for class patterns (fa-xxx, bi bi-xxx, material-icons)

- **Google Fonts Enrichment** (`detectGoogleFonts`):

- Scans CSS `font-family` declarations AND Tailwind `font-[Family_Name]` classes
- Excludes known system fonts, generates Google Fonts `<link>` tags

- **SVG Inline** (`buildSvgDataUri`): Converts `.svg` files to `data:image/svg+xml,...` URIs (URL-encoded, not base64)
- **Mock API Intercept** (`buildMockEndpoints` + runtime script):
 - Maps JSON files to API-like endpoints (`data/users.json` → `/api/users`)
 - Overrides `window.fetch` and `XMLHttpRequest` to serve mock JSON
 - Chose fetch/XHR intercept over Service Worker because `iframe sandbox` limits SW registration
- **Integration:** `buildFrameworkPreview` accepts optional `externalImages?: Map<string,string>` parameter
- `page.tsx` builds `imageAssetMap` from completed `save_image` commands
- All 4 framework builders (React, Vue, Svelte, Vanilla) inject icon CDNs, extra fonts, asset script, and rewrite code paths
- **Zero new dependencies:** All pure TypeScript, static CDN URLs, regex-based detection

Phase A — Code Engine × Skill Factory Integration (2026-05-01):

Phase B — Skill Memory (2026-05-01):

- SkillExecution model in Prisma: tracks every skill execution with `userId`, `skillId`, `method`, `params`, `success`, `duration`, `resultSize`, `error`, `category`, `sessionId`, `trigger` (auto|manual|refine)
- API: `/api/skills/stats` — per-skill stats + global totals
- Persistence: `persistSkillExecution()` in `skill-bridge.ts` (fire-and-forget)
- Dashboard UI: Global stats banner, inline per-skill metrics (count, success %, avg duration, sparkline), expandable recent history
- `lib/skills/skill-bridge.ts` — NEW: `buildSkillContext()`, `executeSkill()`, `extractSkillInvocations()`, `formatSkillFeedback()`
- `lib/skills/code-refiner.ts` — NEW: `analyzeCode()` pre-flight validator (duplicate declarations, brace balance, HTML structure, TS-in-browser, broken imports, score 0-100)
- `lib/claude-code-prompts.ts` — `invoke_skill` in `VALID_ACTIONS` + `BridgeActionType`, §4.5 SKILL SYSTEM in system prompt, skill fields in `normalizeSingle`
- `app/api/arms/claude-code/chat/route.ts` — Skills extracted before transaction, executed server-side, `[SKILL_FEEDBACK]` persisted as system message
- `app/api/arms/claude-code/export/route.ts` — NEW: ZIP export endpoint using archiver
- Code Engine page — Download button in preview toolbar
- Skill Factory page — Code Refiner now `active`
- **Hotfix:** Rewrote §4.5 prompt — skills are `OPTIONAL`, `write_file` is always mandatory
- **Auto-Invoke Image Skill:** Server auto-detects visual projects and injects curated Unsplash URLs as `[IMAGE_CONTEXT]` block into LLM context. New image categories: coffee, restaurant, fitness, realestate, travel. LLM instructed to use those URLs in `<img>` tags.

Phase 6 Hotfix #2 (2026-05-01):

- **Mock endpoint subpath variants:** `buildMockEndpoints()` now registers ALL subpath variants (same as `registerPathVariants` for images). E.g., `focusflow/data/testimonials.json` → also registers `data/testimonials.json`, `./data/testimonials.json`, `/data/testimonials.json`, `testimonials.json`, `/api/testimonials`.
- **Fix button auto-send:** Console “ Fix” button now auto-sends the fix prompt to Octopus instead of just pre-filling. Uses `pendingFixRef` + `useEffect` watching `input` state changes after `sendMessage` is defined (~line 3014).

- **Split asset scripts:** `buildAssetPreScript` (head — mock interceptor) + `buildAssetPostScript` (body — image rewriter). All 4 framework builders updated.

Limitations:

-  Works: React SPA, components, hooks, state, Tailwind, npm packages via esm.sh
-  Works: react-router-dom with Routes/Route/Link navigation
-  Works: HMR — CSS instant swap, JS fast refresh with React state preservation (React only); CSS-only HMR for Vue/Svelte/Vanilla
-  Works: Vue 3 SFCs (template/script/style), `<script setup>`, Options API, vue-router
-  Works: Svelte 4 & 5 components, in-browser compilation
-  Works: Vanilla JS/TS with npm import maps
-  Works: Asset pipeline — images, SVGs, icon CDNs, Google Fonts, mock APIs in all 4 frameworks
-  Limited: Nested layouts, dynamic route params (:id), lazy loading
-  Limited: Full JS HMR only for React — Vue/Svelte/Vanilla get CSS-only HMR (full JS HMR deferred)
-  Not supported: Angular (no in-browser compiler available), Node.js-only packages, SSR

User Preferences / Rafael Notes

- Spanish-first communication. Keep Rafael's tech terminology in Spanish ("Modelo de Imagen", "Motor", etc).
- Fast, decisive workflow — Rafael expects 15-min turnarounds for clear changes.
- Happy with Ad Factory multi-model migration (confirmed working with Vogue/Forbes/Fast Company editorial PR ads).
- Values model transparency — likes to see which model generated each output (badge pattern).
- **Quality benchmark:** Apple × A24 aesthetic. Rejects "startup genérico" look.
- **CRITICAL:** All generic/shared upgrades (like Elite Director) must work for ANY user's product — the platform is multi-tenant. Never hardcode Wildverse features (Scanner/Dimitri/3D) into shared directives.